

ДИСЦИПЛИНА	Программирование в операционной системе Линукс
ИНСТИТУТ	Передовая инженерная школа СВЧ-электроники
КАФЕДРА	Передовых технологий
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания по дисциплине
ПРЕПОДАВАТЕЛЬ	Астафьев Рустам Уралович
СЕМЕСТР	1 семестр, 2025/2026 уч. год

Ссылка на материал:

<https://github.com/astafiev-rustam/programming-in-the-linux-operating-system/tree/lecture-1-3>

Лекция №3: Управление пакетами и программным обеспечением

Теоретическая вводная

Философия управления пакетами в Linux

Представьте, что вы переехали в новый дом и вам нужно обставить его мебелью. В мире Linux есть два основных подхода к этой задаче. Первый — вы идете в гигантский мебельный гипермаркет (репозиторий) и выбираете готовые предметы мебели (пакеты), которые идеально подходят друг к другу и легко собираются. Второй — вы заказываете чертежи у дизайнера (исходный код) и самостоятельно выпиливаете каждую деталь, подгоняя их друг к другу. Оба подхода имеют свои преимущества и используются в разных ситуациях.

Пакет в Linux — это не просто программа. Это тщательно упакованный архив, содержащий не только исполняемые файлы, но и информацию о том, куда их нужно разместить в системе, какие другие пакеты требуются для работы (зависимости), какие конфигурационные файлы создать, и какие действия выполнить до и после установки. Это целая экосистема, которая обеспечивает корректную интеграцию программы в операционную систему.

Две великие династии: DEB и RPM

В мире Linux исторически сложились две основные системы управления пакетами, каждая со своей философией и инструментарией. Система DEB родилась в недрах Debian и распространилась на Ubuntu, Linux Mint и другие производные дистрибутивы. Ее инструменты — **dpkg** для работы с отдельными пакетами и **APT** (Advanced Package Tool) для решения головоломки зависимостей.

С другой стороны, система RPM (Red Hat Package Manager) доминирует в Red Hat Enterprise Linux, CentOS, Fedora и openSUSE. Здесь **rpm** работает с отдельными пакетами, а **yum** и его современная версия **dnf** управляют зависимостями и репозиториями. Интересно, что обе системы в конечном счете решают одни и те же задачи, но делают это немного по-разному, как два шеф-повара, готовящие одно и то же блюдо по разным рецептам.

Репозитории: общественные склады программ

Репозитории — это фундаментальное понятие в экосистеме Linux. Представьте огромную библиотеку, где каждая книга (пакет) прошла проверку на качество, совместимость и безопасность. Система знает адреса этих библиотек и может автоматически находить там нужные программы, проверять их подлинность с помощью цифровых подписей и загружать вместе со всеми зависимостями.

Каждый дистрибутив поддерживает свои официальные репозитории, но также позволяет добавлять сторонние хранилища. Вот где в игру вступают GPG-ключи — цифровые подписи, которые гарантируют, что пакеты поступили из доверенного источника и не были изменены злоумышленниками. Без правильного ключа система просто откажется устанавливать пакеты из такого репозитория, защищая вас от потенциальных угроз.

Зависимости: паутина взаимосвязей

Одна из самых элегантных и одновременно сложных концепций в управлении пакетами — это зависимости. Современное программное обеспечение редко работает в вакууме. Программам нужны библиотеки, движки, сервисы — как инструменты нуждаются в мастерской, а музыканты в оркестре. Пакетный менеджер автоматически находит все необходимые компоненты и устанавливает их вместе с основной программой.

Но иногда эта паутина зависимость становится настолько сложной, что возникает "dependency hell" — ад зависимостей, когда разные программы требуют несовместимые версии одних и тех же библиотек. Современные пакетные менеджеры научились справляться с этой проблемой, но понимание механизма зависимости остается критически важным для любого системного администратора или разработчика.

Сборка из исходников: путь самурая

Несмотря на все удобства пакетных менеджеров, иногда возникает необходимость собрать программу из исходного кода. Это может потребоваться, когда нужна самая свежая версия программы, которую еще не добавили в репозитории, или когда нужно включить специфические опции сборки, или просто для обучения.

Процесс обычно следует классической трилогии: `./configure` проверяет систему на наличие всех необходимых компонентов и настраивает сборку под конкретное окружение; `make` компилирует тысячи строк кода в исполняемые файлы; `make install` аккуратно размещает полученные файлы в нужных каталогах системы. Это более трудоемкий путь, но он дает полный контроль над тем, что и как устанавливается в вашу систему.

Эволюция упаковки: контейнеры и универсальные пакеты

В последние годы традиционные системы пакетов получили развитие в виде таких технологий, как Snap, Flatpak и AppImage. Эти системы создают изолированные среды, где программы работают вместе со всеми своими зависимостями, что решает проблему конфликта версий и упрощает установку. Они похожи на готовые мебельные гарнитуры, которые можно поставить в любой комнате, не беспокоясь о совместимости с существующим интерьером.

Практические примеры

Пример 1: Первое знакомство с APT

Цель: Научиться основам работы с APT в Debian/Ubuntu.

```
# 1. Обновляем информацию о доступных пакетах из репозиториев
# APT хранит локальную базу данных о пакетах, которую нужно периодически обновлять
sudo apt update

# 2. Просмотрим список обновляемых пакетов
apt list --upgradable

# 3. Обновляем все установленные пакеты до последних версий
sudo apt upgrade

# 4. Ищем пакет по названию (например, текстовый редактор vim)
apt search vim

# 5. Показываем подробную информацию о пакете
apt show vim

# 6. Устанавливаем пакет
sudo apt install vim

# 7. Проверяем, что пакет установился
which vim
vim --version
```

Пример 2: Работа с DNF в Fedora/CentOS

Цель: Освоить базовые операции с DNF в RPM-дистрибутивах.

```
# 1. Обновляем информацию о пакетах (аналог apt update)
sudo dnf check-update

# 2. Устанавливаем пакет (например, midnight commander)
sudo dnf install mc

# 3. Ищем пакеты, связанные с python
dnf search python

# 4. Показываем информацию о пакете
dnf info python3

# 5. Обновляем все пакеты в системе
sudo dnf update

# 6. Удаляем пакет
sudo dnf remove mc

# 7. Просматриваем историю операций
dnf history
```

Пример 3: Управление репозиториями

Цель: Научиться добавлять и управлять репозиториями.

```
# 1. Просмотрим список всех доступных репозиториях в Ubuntu
ls -la /etc/apt/sources.list.d/
cat /etc/apt/sources.list

# 2. Добавим репозиторий с популярным ПО (на примере Chrome)
wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | sudo apt-key add -
sudo sh -c 'echo "deb [arch=amd64] http://dl.google.com/linux/chrome/deb/ stable
main" >> /etc/apt/sources.list.d/google-chrome.list'

# 3. Обновим информацию о пакетах с учетом нового репозитория
sudo apt update

# 4. Теперь можем установить Chrome из добавленного репозитория
# sudo apt install google-chrome-stable

# 5. В Fedora/CentOS репозитории добавляются по-другому
# sudo dnf config-manager --add-repo https://example.com/repo.repo

# 6. Удалим добавленный репозиторий
sudo rm /etc/apt/sources.list.d/google-chrome.list
sudo apt update
```

Пример 4: Поиск и установка специфических версий пакетов

Цель: Научиться работать с конкретными версиями пакетов.

```
# 1. Просмотрим все доступные версии пакета python3
apt-cache policy python3

# 2. Или альтернативный способ
apt-cache show python3 | grep Version

# 3. Установим конкретную версию пакета (если доступна)
# sudo apt install python3=3.8.2-0ubuntu2

# 4. Зафиксируем версию пакета, чтобы она не обновлялась
sudo apt-mark hold python3

# 5. Проверим, какие пакеты зафиксированы
apt-mark showhold

# 6. Разрешим обновление пакета
sudo apt-mark unhold python3
```

```
# 7. В DNF просмотр доступных версий
dnf --showduplicates list python3
```

Пример 5: Удаление пакетов и очистка системы

Цель: Научиться правильно удалять пакеты и чистить систему.

```
# 1. Удаляем пакет, но оставляем конфигурационные файлы
sudo apt remove vim

# 2. Удаляем пакет полностью с конфигурационными файлами
sudo apt purge vim

# 3. Автоматически удаляем пакеты, которые больше не нужны
sudo apt autoremove

# 4. Очищаем кеш загруженных пакетов
sudo apt clean

# 5. Очищаем кеш, но оставляем последние версии пакетов
sudo apt autoclean

# 6. В DNF полное удаление пакета
sudo dnf remove vim
sudo dnf autoremove

# 7. Просмотр размера кеша DNF
sudo dnf clean dbcache
```

Пример 6: Работа с пакетами в офлайн-режиме

Цель: Научиться устанавливать пакеты без доступа к интернету.

```
# 1. Скачаем пакет и все его зависимости для офлайн-установки
apt-get download vim
apt-cache depends vim | grep Depends | awk '{print $2}' | xargs apt-get download

# 2. Установим скачанные .deb пакеты вручную
sudo dpkg -i *.deb

# 3. Если возникли проблемы с зависимостями
sudo apt-get install -f

# 4. В DNF скачивание пакетов для офлайн-установки
dnf download vim
dnf --downloadonly install vim

# 5. Создаем локальный репозиторий из скачанных пакетов
mkdir local-repo
```

```
mv *.deb local-repo/  
cd local-repo  
dpkg-scanpackages . /dev/null | gzip -9c > Packages.gz  
  
# 6. Добавляем локальный репозиторий  
echo "deb [trusted=yes] file:$(pwd) ./" | sudo tee /etc/apt/sources.list.d/local-repo.list  
sudo apt update
```

Пример 7: Анализ установленных пакетов

Цель: Научиться анализировать и управлять установленными пакетами.

```
# 1. Просмотрим все установленные пакеты  
dpkg -l  
  
# 2. Ищем конкретный установленный пакет  
dpkg -l | grep python  
  
# 3. Показываем файлы, установленные пакетом  
dpkg -L vim  
  
# 4. Определяем, какому пакету принадлежит файл  
dpkg -S /bin/ls  
  
# 5. Просмотр истории операций apt  
cat /var/log/apt/history.log  
  
# 6. В DNF аналогичные операции  
dnf list installed  
rpm -qa | grep python  
rpm -ql vim  
rpm -qf /bin/ls
```

Пример 8: Сборка пакета из исходных кодов

Цель: Научиться собирать и устанавливать программу из исходного кода.

```
# 1. Установим инструменты для сборки  
sudo apt install build-essential devscripts fakeroot  
  
# 2. Создадим тестовую программу  
cat > hello.c << 'EOF'  
#include <stdio.h>  
int main() {  
    printf("Hello, Package Management!\n");  
    return 0;  
}  
EOF
```

```
# 3. Скомпилируем программу
gcc -o hello hello.c

# 4. Проверим работу
./hello

# 5. Создаем простой Makefile для управления сборкой
cat > Makefile << 'EOF'
PREFIX=/usr/local

all: hello

hello: hello.c
    gcc -o hello hello.c

install:
    install -D -m 0755 hello $(DESTDIR)$(PREFIX)/bin/hello

clean:
    rm -f hello

.PHONY: all install clean
EOF

# 6. Устанавливаем программу в систему
sudo make install

# 7. Проверяем установку
which hello
hello
```

Пример 9: Создание простого DEB-пакета

Цель: Научиться создавать собственные DEB-пакеты.

```
# 1. Создаем структуру каталогов для пакета
mkdir -p myhello/DEBIAN myhello/usr/bin myhello/usr/share/doc/myhello

# 2. Копируем нашу программу
cp hello myhello/usr/bin/

# 3. Создаем файл контроля пакета
cat > myhello/DEBIAN/control << 'EOF'
Package: myhello
Version: 1.0-1
Section: utils
Priority: optional
Architecture: amd64
Depends: libc6 (>= 2.34)
Maintainer: Your Name <your.email@example.com>
```

```
Description: A simple hello world program
This is a test package that provides a simple
hello world program for educational purposes.
EOF

# 4. Создаем скрипт пост-установки
cat > myhello/DEBIAN/postinst << 'EOF'
#!/bin/bash
echo "MyHello package was successfully installed!"
EOF
chmod 755 myhello/DEBIAN/postinst

# 5. Собираем пакет
dpkg-deb --build myhello

# 6. Устанавливаем созданный пакет
sudo dpkg -i myhello.deb

# 7. Проверяем установку
myhello

# 8. Удаляем пакет
sudo dpkg -r myhello
```

Пример 10: Сборка пакета из исходников с autotools

Цель: Освоить классический способ сборки программ.

```
# 1. Установим необходимые инструменты
sudo apt install autoconf automake libtool

# 2. Создадим простой проект с autotools
mkdir myproject
cd myproject

# 3. Создаем исходный код
cat > hello.c << 'EOF'
#include <stdio.h>
#include "config.h"

int main() {
    printf("Hello, %s!\n", PACKAGE_STRING);
    return 0;
}
EOF

# 4. Создаем configure.ac
cat > configure.ac << 'EOF'
AC_INIT([hello], [1.0], [your@email.com])
AM_INIT_AUTOMAKE
AC_PROG_CC
```

```
AC_CONFIG_HEADERS([config.h])
AC_CONFIG_FILES([Makefile])
AC_OUTPUT
EOF

# 5. Создаем Makefile.am
cat > Makefile.am << 'EOF'
bin_PROGRAMS = hello
hello_SOURCES = hello.c
EOF

# 6. Генерируем скрипты конфигурации
autoreconf --install

# 7. Настраиваем сборку
./configure --prefix=/usr/local

# 8. Собираем программу
make

# 9. Устанавливаем (опционально)
sudo make install
```

Пример 11: Работа с Snap пакетами

Цель: Познакомиться с современной системой универсальных пакетов.

```
# 1. Устанавливаем Snap (если не установлен)
sudo apt update
sudo apt install snapd

# 2. Ищем пакеты в Snap Store
snap find hello

# 3. Устанавливаем Snap пакет
sudo snap install hello-world

# 4. Запускаем установленное приложение
hello-world

# 5. Просматриваем установленные Snap пакеты
snap list

# 6. Обновляем Snap пакеты
sudo snap refresh

# 7. Удаляем Snap пакет
sudo snap remove hello-world

# 8. Просматриваем информацию о пакете
snap info chromium
```

Пример 12: Работа с Flatpak

Цель: Освоить альтернативную систему универсальных пакетов.

```
# 1. Устанавливаем Flatpak
sudo apt install flatpak

# 2. Добавляем репозиторий Flathub
flatpak remote-add --if-not-exists flathub
https://flathub.org/repo/flathub.flatpakrepo

# 3. Ищем приложения
flatpak search gimp

# 4. Устанавливаем приложение
flatpak install flathub org.gimp.GIMP

# 5. Запускаем установленное приложение
flatpak run org.gimp.GIMP

# 6. Просматриваем установленные приложения
flatpak list

# 7. Обновляем приложения
flatpak update

# 8. Удаляем приложение
flatpak uninstall org.gimp.GIMP
```

Пример 13: Диагностика проблем с пакетами

Цель: Научиться решать типичные проблемы с пакетами.

```
# 1. Проверяем целостность пакетной базы
sudo dpkg --configure -a
sudo apt --fix-broken install

# 2. Восстанавливаем поврежденные пакеты
sudo apt install --reinstall package-name

# 3. Очищаем заблокированные файлы
sudo rm /var/lib/dpkg/lock
sudo rm /var/lib/apt/lists/lock

# 4. Проверяем зависимости
apt-get check

# 5. Восстанавливаем базу данных RPM
sudo rpm --rebuilddb
```

```
# 6. Проверяем целостность пакета
rpm -V package-name

# 7. Анализируем конфликтующие файлы
dpkg -S /path/to/conflicting/file
```

Пример 14: Мониторинг и логирование пакетных операций

Цель: Научиться отслеживать операции с пакетами.

```
# 1. Просматриваем историю операций APT
cat /var/log/apt/history.log

# 2. Смотрим детальные логи APT
cat /var/log/apt/term.log

# 3. Устанавливаем утилиту для мониторинга изменений в файловой системе
sudo apt install apt-listchanges

# 4. Настраиваем отправку уведомлений об обновлениях
sudo apt install apticron

# 5. Просматриваем историю DNF
dnf history list

# 6. Откатываем операцию в DNF
sudo dnf history undo <ID>

# 7. Мониторим использование диска пакетами
dpkg-query -Wf '${Installed-Size}\t${Package}\n' | sort -nr | head -10
```

Пример 15: Создание локального зеркала репозитория

Цель: Научиться создавать локальное зеркало для ускорения установки пакетов.

```
# 1. Устанавливаем инструменты для создания зеркала
sudo apt install apt-mirror

# 2. Создаем конфигурационный файл
sudo mkdir -p /etc/apt/mirror.list.d
cat > /etc/apt/mirror.list << 'EOF'
set base_path /var/spool/apt-mirror
set nthreads 20
set _tilde 0

deb http://archive.ubuntu.com/ubuntu jammy main restricted universe multiverse
deb http://archive.ubuntu.com/ubuntu jammy-security main restricted universe
multiverse
```

```
deb http://archive.ubuntu.com/ubuntu jammy-updates main restricted universe
multiverse

clean http://archive.ubuntu.com/ubuntu
EOF

# 3. Запускаем зеркалирование
sudo apt-mirror

# 4. Настраиваем веб-сервер для раздачи зеркала
sudo apt install nginx
sudo ln -s /var/spool/apt-mirror/mirror/archive.ubuntu.com/ubuntu
/var/www/html/ubuntu

# 5. Настраиваем клиенты для использования локального зеркала
# На клиентских машинах:
# sudo sed -i 's|http://archive.ubuntu.com/ubuntu|http://your-mirror-
server/ubuntu|g' /etc/apt/sources.list

# 6. Создаем скрипт для автоматического обновления зеркала
cat > /etc/cron.d/apt-mirror << 'EOF'
0 4 * * * apt-mirror /usr/bin/apt-mirror > /var/spool/apt-mirror/var/cron.log
EOF
```